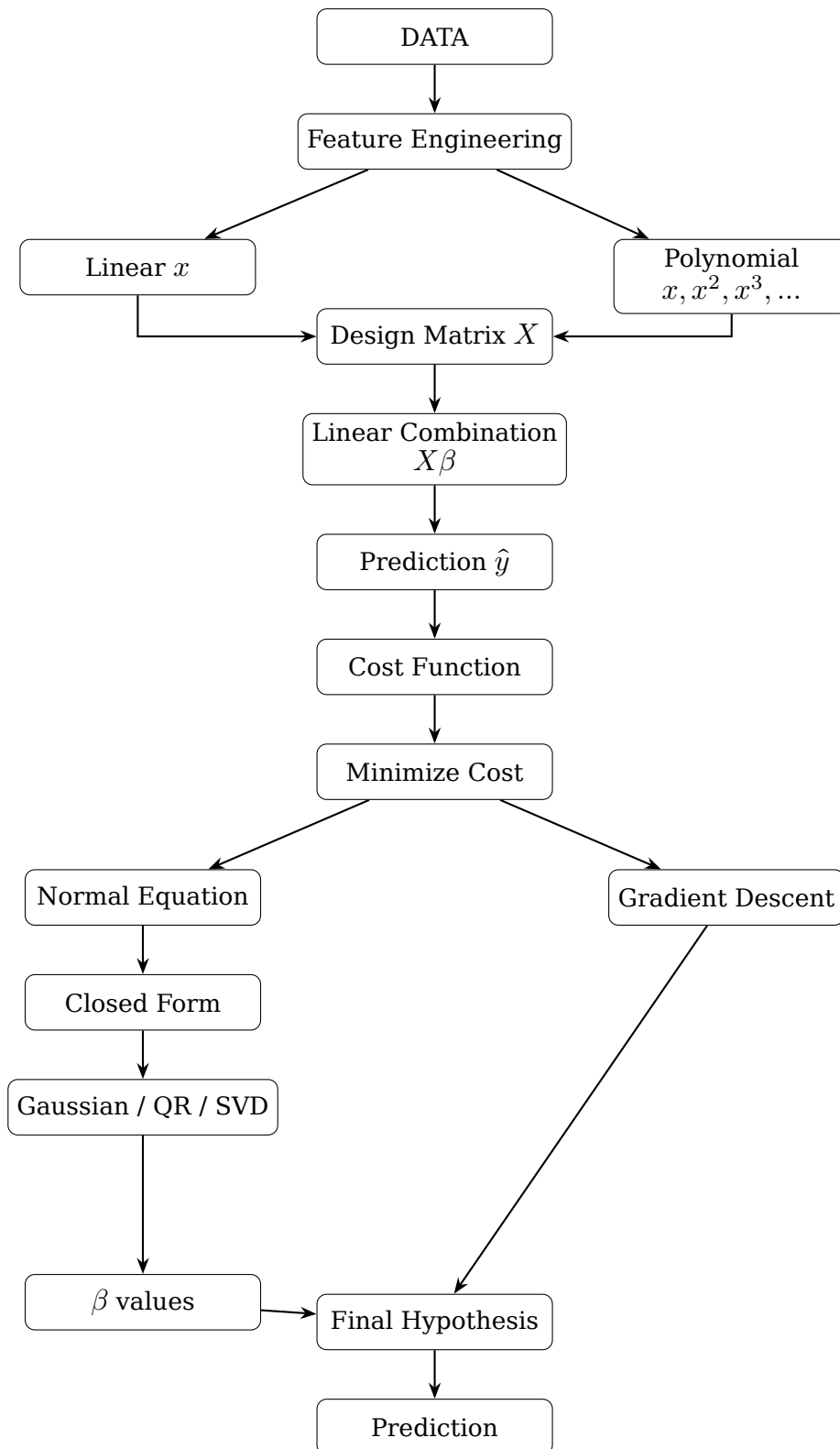


Feature Engineering & Polynomial Regression — Full Derivation From Scratch

Contents

Overview Flowchart	2
Step 1 — Data	4
Step 2 — Feature Engineering	4
Step 3 — Design Matrix X	4
Step 4 — Parameter Vector and Linear Combination	5
Step 5 — Prediction $\hat{y}$	5
Step 6 — Cost Function	5
Step 7 — Minimize the Cost	6
Step 8 — Compute $X^T X$ and $X^T y$	6
Step 9 — Method 1: Solve via Gaussian Elimination	7
Step 10 — Method 2: Solve via QR Decomposition	8
Step 11 — Method 3: Solve via SVD (Singular Value Decomposition)	9
Step 12 — Interpreting $\beta_2 = 0$	9
Step 13 — Final Hypothesis	10
Step 14 — Prediction	10
Step 15 — When to Use Polynomial Regression	10
Summary Table	11

Overview Flowchart



This document works through every box above using one concrete dataset — including

fitting a **polynomial** (quadratic) model and solving it three different ways: Gaussian elimination, QR decomposition, and SVD.

Step 1 — Data

Same base dataset as before:

$$(1, 2), \quad (2, 4), \quad (3, 5), \quad (4, 7)$$

$$x = [1, 2, 3, 4]^T, \quad y = [2, 4, 5, 7]^T, \quad n = 4$$

Terms: data point, sample size n .

Step 2 — Feature Engineering

Feature engineering means constructing the input columns fed into the model — it's a choice you make *before* fitting, not something the algorithm decides on its own.

Option A — Linear feature. Use x as-is:

$$\text{features} = [1, x]$$

Option B — Polynomial features. Add higher powers of x as extra columns, letting the same linear-algebra machinery fit a *curved* relationship:

$$\text{features} = [1, x, x^2, x^3, \dots]$$

Critically: the model is still **linear in its parameters** β even though it's non-linear in x — $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$ is a straight-line combination of the columns $1, x, x^2$, just as $\hat{y} = \beta_0 + \beta_1 x$ was a combination of $1, x$. This is why everything from the earlier normal-equation derivation still applies unchanged.

Our worked example uses Option B with one quadratic term (x^2), to show the general mechanics:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

Terms: feature engineering, polynomial features, linear-in-parameters.

Step 3 — Design Matrix X

Build one column per feature: intercept (1's), x , and x^2 .

x	x^2
1	1
2	4
3	9
4	16

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \end{bmatrix}$$

Terms: design matrix, polynomial feature column.

Step 4 — Parameter Vector and Linear Combination

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$$\hat{y} = X\beta = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} \beta_0 + \beta_1 + \beta_2 \\ \beta_0 + 2\beta_1 + 4\beta_2 \\ \beta_0 + 3\beta_1 + 9\beta_2 \\ \beta_0 + 4\beta_1 + 16\beta_2 \end{bmatrix}$$

Each row is a linear combination of the three columns of X .

Terms: parameter vector, linear combination.

Step 5 — Prediction $\hat{y}$

$\hat{y}$ (from Step 4) is exactly the model's predicted output for each row — same meaning as in the linear case, just with one more term per row.

Step 6 — Cost Function

$$J(\beta) = \frac{1}{2n}(y - X\beta)^T(y - X\beta)$$

Identical formula to the linear case — the cost function doesn't care how many columns X has, only that the model is linear in β .

Terms: cost function, quadratic form.

Step 7 — Minimize the Cost

Exactly as before (see the closed-form derivation notes for the full calculus), setting $\frac{\partial J}{\partial \beta} = 0$ gives the **normal equation**:

$$X^T X \beta = X^T y$$

Two ways to solve this:

- **Direct (closed form):** solve the normal equation exactly. Several numerically different but mathematically equivalent routes exist: Gaussian elimination, QR decomposition, SVD.
- **Iterative:** gradient descent, updating β repeatedly (same update rule as before, just with 3 parameters instead of 2 — not re-derived here since the mechanics are identical to the earlier gradient descent document, just with an extra β_2 term and an extra x^2 factor in its gradient).

Terms: normal equation, closed-form, iterative.

Step 8 — Compute $X^T X$ and $X^T y$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 1 & 4 & 9 & 16 \end{bmatrix}$$

$X^T X$ (3×3), entry by entry:

- (1, 1): $1 + 1 + 1 + 1 = 4$
- (1, 2): $1 + 2 + 3 + 4 = 10$
- (1, 3): $1 + 4 + 9 + 16 = 30$
- (2, 2): $1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$
- (2, 3): $1(1) + 2(4) + 3(9) + 4(16) = 1 + 8 + 27 + 64 = 100$
- (3, 3): $1^2 + 4^2 + 9^2 + 16^2 = 1 + 16 + 81 + 256 = 354$

(Symmetric, so $(2, 1) = (1, 2)$, $(3, 1) = (1, 3)$, $(3, 2) = (2, 3)$.)

$$X^T X = \begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix}$$

$X^T y$ (3×1):

- Row 1: $2 + 4 + 5 + 7 = 18$
- Row 2: $1(2) + 2(4) + 3(5) + 4(7) = 2 + 8 + 15 + 28 = 53$
- Row 3: $1(2) + 4(4) + 9(5) + 16(7) = 2 + 16 + 45 + 112 = 175$

$$X^T y = \begin{bmatrix} 18 \\ 53 \\ 175 \end{bmatrix}$$

Step 9 — Method 1: Solve via Gaussian Elimination

The normal equation as a 3×3 system:

$$4\beta_0 + 10\beta_1 + 30\beta_2 = 18 \quad (\text{I})$$

$$10\beta_0 + 30\beta_1 + 100\beta_2 = 53 \quad (\text{II})$$

$$30\beta_0 + 100\beta_1 + 354\beta_2 = 175 \quad (\text{III})$$

Eliminate β_0 from (II): multiply (I) by $10/4 = 2.5$: $10\beta_0 + 25\beta_1 + 75\beta_2 = 45$. Subtract from (II):

$$(10\beta_0 + 30\beta_1 + 100\beta_2) - (10\beta_0 + 25\beta_1 + 75\beta_2) = 53 - 45$$

$$5\beta_1 + 25\beta_2 = 8 \quad (\text{II}')$$

Eliminate β_0 from (III): multiply (I) by $30/4 = 7.5$: $30\beta_0 + 75\beta_1 + 225\beta_2 = 135$. Subtract from (III):

$$(30\beta_0 + 100\beta_1 + 354\beta_2) - (30\beta_0 + 75\beta_1 + 225\beta_2) = 175 - 135$$

$$25\beta_1 + 129\beta_2 = 40 \quad (\text{III}')$$

Eliminate β_1 from (III'): multiply (II') by $25/5 = 5$: $25\beta_1 + 125\beta_2 = 40$. Subtract from (III'):

$$(25\beta_1 + 129\beta_2) - (25\beta_1 + 125\beta_2) = 40 - 40$$

$$4\beta_2 = 0 \implies \beta_2 = 0$$

Back-substitute into (II'): $5\beta_1 + 25(0) = 8 \implies \beta_1 = \frac{8}{5} = 1.6$

Back-substitute into (I): $4\beta_0 + 10(1.6) + 30(0) = 18 \implies 4\beta_0 + 16 = 18 \implies \beta_0 = 0.5$

$$\boxed{\beta_0 = 0.5, \quad \beta_1 = 1.6, \quad \beta_2 = 0}$$

Terms: forward elimination, back-substitution.

Step 10 — Method 2: Solve via QR Decomposition

Idea. Factor $X = QR$, where Q has orthonormal columns ($Q^T Q = I$) and R is upper triangular. Substituting into the normal equation:

$$X^T X \beta = X^T y \implies (QR)^T (QR) \beta = (QR)^T y \implies R^T Q^T Q R \beta = R^T Q^T y$$

Since $Q^T Q = I$:

$$R^T R \beta = R^T Q^T y$$

Assuming R is invertible, cancel R^T from both sides:

$$\boxed{R \beta = Q^T y}$$

This avoids ever forming $X^T X$ directly — one reason QR is more numerically stable than the raw normal equation.

Computed factors (from decomposing our X):

$$Q \approx \begin{bmatrix} -0.5 & 0.6708 & 0.5 \\ -0.5 & 0.2236 & -0.5 \\ -0.5 & -0.2236 & -0.5 \\ -0.5 & -0.6708 & 0.5 \end{bmatrix}, \quad R \approx \begin{bmatrix} -2 & -5 & -15 \\ 0 & -2.2361 & -11.1803 \\ 0 & 0 & 2 \end{bmatrix}$$

Compute $Q^T y$:

$$Q^T y \approx \begin{bmatrix} -9 \\ -3.5777 \\ 0 \end{bmatrix}$$

Solve $R \beta = Q^T y$ by back-substitution (since R is upper triangular):

Row 3: $2\beta_2 = 0 \implies \beta_2 = 0$

Row 2: $-2.2361\beta_1 - 11.1803\beta_2 = -3.5777 \implies -2.2361\beta_1 = -3.5777 \implies \beta_1 = 1.6$

Row 1: $-2\beta_0 - 5\beta_1 - 15\beta_2 = -9 \implies -2\beta_0 - 5(1.6) = -9 \implies -2\beta_0 - 8 = -9 \implies -2\beta_0 = -1 \implies \beta_0 = 0.5$

$$\beta_0 = 0.5, \quad \beta_1 = 1.6, \quad \beta_2 = 0$$

Matches Gaussian elimination exactly.

Terms: QR decomposition, orthonormal columns, upper triangular matrix, back-substitution.

Step 11 — Method 3: Solve via SVD (Singular Value Decomposition)

Idea. Factor $X = U\Sigma V^T$, where U, V have orthonormal columns and Σ is diagonal with the **singular values** of X . This is the most numerically robust method — it works even when $X^T X$ is singular or near-singular, by using the **pseudo-inverse**:

$$\beta = V\Sigma^{-1}U^T y \quad (\text{i.e. } \beta = X^+ y, \text{ the Moore-Penrose pseudo-inverse solution})$$

Computed singular values for our X :

$$\Sigma \approx \text{diag}(19.621, 1.712, 0.266)$$

All three are non-zero, confirming X has full column rank (consistent with Step 9's Gaussian-elimination pivots all being non-zero). Carrying out the matrix multiplication $V\Sigma^{-1}U^T y$ with the computed U, Σ, V gives:

$$\beta_0 = 0.5, \quad \beta_1 = 1.6, \quad \beta_2 = 0$$

Matches both previous methods exactly.

Terms: singular value decomposition (SVD), singular values, pseudo-inverse, full column rank.

Step 12 — Interpreting $\beta_2 = 0$

All three solution methods agree: the quadratic coefficient β_2 comes out to (numerically) zero. This is not a computational accident — it's the algorithm correctly detecting that **the underlying data has no real curvature**: the same four points were exactly fit by a straight line in the earlier linear-regression document. Adding the x^2 feature gave the model the *option* to curve, and the closed-form solution chose not to use it, because a flat coefficient there minimizes the cost.

This illustrates a general point about feature engineering: **adding a feature never forces the model to use it** — the normal equation (or gradient descent) will assign it a near-zero coefficient if it doesn't help reduce the cost.

Step 13 — Final Hypothesis

$$\hat{y} = 0.5 + 1.6x + 0 \cdot x^2 = 0.5 + 1.6x$$

Reduces to exactly the same regression line found in the linear-only case.

Step 14 — Prediction

For the same new input $x = 5$ used throughout the earlier documents, using $x^2 = 25$ as well:

$$\hat{y} = 0.5 + 1.6(5) + 0(25) = 0.5 + 8.0 + 0 = \boxed{8.5}$$

Identical to the linear model's prediction — confirming that feature engineering, when the extra features aren't needed, doesn't change the outcome; it only adds the *capacity* for a better fit if the data had actually called for it.

Step 15 — When to Use Polynomial Regression

Polynomial features are a tool, not a default. Here's how to judge when they help versus when they hurt.

Use polynomial regression when:

- **The scatter plot shows curvature.** If plotting y against x reveals a bend, hump, or S-shape that a straight line clearly can't follow, a linear model will systematically **underfit** — the residuals will show an obvious pattern (e.g. all positive in the middle, negative at the ends) instead of being randomly scattered.
- **Domain knowledge suggests a non-linear relationship.** E.g. projectile motion follows x^2 terms (physics), diminishing returns often follow a log or fractional-power shape, and growth processes may need higher-order or exponential terms.
- **A linear model's residual plot shows a curved pattern.** Even without a clear scatter-plot bend, plotting residuals (e_i) against x and seeing a curve (not a random cloud) signals a missing non-linear term.

- **You have enough data.** Every extra feature (degree) is a new parameter to estimate; you need more data points than parameters, and ideally several times more, or the fit becomes unreliable.

Avoid or limit polynomial regression when:

- **The relationship already looks linear** (as in this document's dataset) — adding $x^2, x^3, \dots$ just adds noise-fitting capacity without improving the true fit, as Step 12 demonstrated with $\beta_2 = 0$.
- **The degree is high relative to the data size.** A high-degree polynomial can pass through every training point almost exactly (near-zero training cost) while wiggling wildly between points — classic **overfitting**. It memorizes noise instead of learning the trend.
- **Extrapolation is required.** Polynomials of degree 2+ often shoot off sharply outside the training range (unlike a straight line's steady slope), making predictions far from the observed x values unreliable — even more so than the linear extrapolation caution noted in Step 14.
- **Multicollinearity becomes severe.** x, x^2, x^3 are often highly correlated with each other over a narrow x -range, which can make $X^T X$ nearly singular — this is exactly the numerical-stability problem that QR and SVD (Steps 10–11) are more robust against than a raw normal-equation inverse.

Practical rule of thumb: start with the simplest model (linear). Only add polynomial terms if the residual pattern or domain knowledge justifies it, check the fit on **held-out data** (not just training data) to catch overfitting, and prefer the lowest degree that removes the systematic pattern in the residuals — going from degree 1 to 2 is common; degree 4+ is rarely justified without a strong theoretical reason.

Terms: underfitting, overfitting, residual plot, held-out/validation data, degree selection, bias-variance trade-off.

Summary Table

Order	Concept	Result
2	Feature engineering choice	Polynomial features $[1, x, x^2]$
3	Design matrix X	4×3 matrix with columns $1, x, x^2$
8	$X^T X$	$\begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix}$
8	$X^T y$	$\begin{bmatrix} 18 \\ 53 \\ 175 \end{bmatrix}$
9	Gaussian elimination result	$\beta_0 = 0.5, \beta_1 = 1.6, \beta_2 = 0$

Order	Concept	Result
10	QR decomposition result	$\beta_0 = 0.5, \beta_1 = 1.6, \beta_2 = 0$
11	SVD result	$\beta_0 = 0.5, \beta_1 = 1.6, \beta_2 = 0$
13	Final hypothesis	$\hat{y} = 0.5 + 1.6x$
14	Prediction at $x = 5$	$\hat{y} = 8.5$
15	When to use it	Curved data, domain justification, enough data, watch for overfitting/extrapolation